Distributed Financial Transaction Processor

Individual Final Report

Sampath Pranay Beela
CS6650 — Building Scalable Distributed Systems

Spring 2026

Abstract

This report documents my contributions to the team project for CS6650 — a distributed financial transaction processor that accepts money-movement requests over HTTP, queues them through Amazon SQS, and applies them atomically against account balances stored in Amazon DynamoDB. Across eight weeks of engineering, my responsibilities covered system architecture finalization, repository and service scaffolding, the end-to-end transfer workflow, SQS integration, containerization for AWS Fargate, cloud smoke testing, the Locust-based load-testing framework, Experiment 1 (concurrent transfer storm against a hot account), my share of Experiment 3 (optimistic vs. pessimistic comparison) and Experiment 4 (horizontal scaling), the data analysis and chart generation pipeline, and demo preparation. The system runs exclusively on AWS, provisioned through Terraform, with `LabRole` as the sole principal attached to every ECS task. This report walks through each of my contributions, the experiments I ran, the numbers I collected, and every non-trivial obstacle I worked through along the way.

Contents

1 Introduction and Scope

The project team was split along two tracks. My track covered the public-facing and load-generation surface of the system: the HTTP API, the queue integration, packaging for deployment, the load test harness, and the experiments that stress the system as a whole. This report therefore focuses on the items in my work list from the project implementation plan:

1. Project scope and architecture finalization (Weeks 1–2)
2. Repository and service scaffolding (Weeks 1–2)
3. Core transfer workflow implementation (Weeks 1–2)
4. SQS integration (Weeks 1–2)
5. Containerization (Weeks 3–4)
6. Cloud smoke testing (Weeks 3–4)
7. Load testing framework (Weeks 3–4)
8. Experiment 1: concurrent transfer storm (Weeks 3–4)
9. Experiment 3: optimistic vs. pessimistic comparison (Weeks 5–6)
10. Experiment 4: horizontal scaling verification (Weeks 5–6)
11. Data analysis and chart generation (Weeks 7–8)
12. Final demo preparation (Weeks 7–8)

Other items from the plan (DynamoDB schema design, idempotency support at the API layer, local correctness verification, Terraform infrastructure authorship, the pessimistic locking implementation, the worker crash-recovery experiment, and final report writing) were owned by my collaborator. I refer to those components where they are essential for understanding the system as a whole, but I do not claim design or implementation ownership over them.

2 Architecture Finalization

Before any code was written we needed a firm agreement on the shape of the system, the network path for each request, and the responsibility boundary between the API service and the asynchronous worker. I drove the architecture review and finalized the diagram we used for the rest of the project (Figure 1).

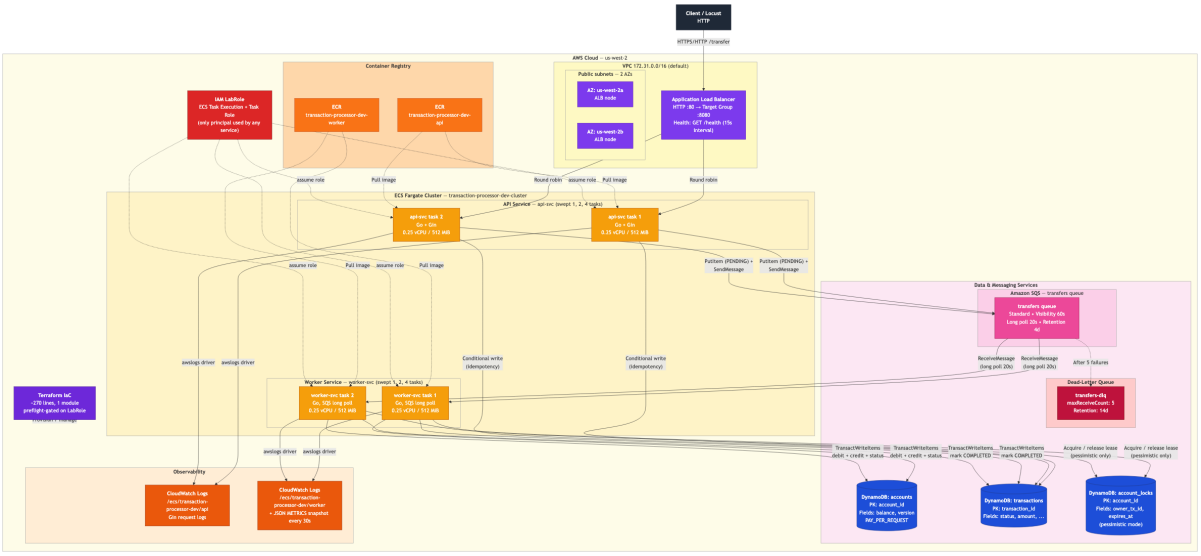


Figure 1: Deployed architecture. Every arrow crosses an AWS service boundary; no component runs locally. Every ECS task runs under LabRole; the stack creates no IAM resources of its own. Terraform provisions everything inside the yellow AWS Cloud container.

The guiding decisions that came out of the architecture review were:

- **Asynchronous by default.** The API returns HTTP 202 the moment a transfer is accepted and enqueued. All balance mutation happens in the worker path. This lets us scale API and worker fleets independently and gives us a natural crash-safety story through queue redelivery.
- **One atomic commit per transfer.** Debit, credit, and transaction-status update are bundled into a single DynamoDB `TransactWriteItems` call. A mid-commit crash cannot leave money half-moved.
- **Cloud-only.** There would be no permanent local-compose workflow. All correctness and performance numbers in the final report would be collected against real AWS services (ALB, ECS Fargate, SQS, DynamoDB, CloudWatch).
- **LabRole everywhere.** The only IAM principal wired into ECS task execution and task roles is `LabRole`. Terraform creates no IAM resources.

3 Repository and Service Scaffolding

I laid out the repository so that the API service, the worker service, infrastructure, and test harnesses each live in their own top-level directory while sharing a single Go module. The resulting structure (after all work was completed) is shown in Listing 1.

```
transaction-processor/
  api-svc/            # HTTP API: handlers, queue client, dynamo client, main.
      go
    Dockerfile
    handlers/         # /transfer (POST), /transfer/:id (GET), /health
    queue/            # SQS send wrapper
    db/               # DynamoDB accessors
    models/           # request/response and Transaction types
  worker-svc/         # Asynchronous transfer processor
    Dockerfile
    processor/        # optimistic + pessimistic paths
    queue/            # SQS receive/delete wrapper
    db/               # DynamoDB accessors, lock primitives
```

```
    metrics/          # prom-style counters + /metrics endpoint
  load-tests/         # Locust workloads + verify_balances.py
  scripts/            # seed, push-images, smoke, experiment runners, charts
  infra/              # Terraform stack (ECR, ECS, ALB, SQS, DynamoDB,
      CloudWatch)
  Makefile            # cloud-only targets, each preflight-gated
```

Listing 1: Repository layout.

3.1 API service skeleton

I wrote the API service in Go using the Gin framework. The HTTP surface exposes three routes:

```
r := gin.Default()
r.GET("/health", handlers.HealthCheck)
r.POST("/transfer", h.PostTransfer)
r.GET("/transfer/:id", h.GetTransfer)
```

Listing 2: Route registration in `api-svc/main.go`.

`/health` is the ALB's target-group health probe. `POST /transfer` accepts a JSON body with a client-supplied `transaction_id` for idempotency; `GET /transfer/:id` lets the client poll the outcome. The `PostTransfer` handler validates the body, writes the pending record, and enqueues the message — the three steps run sequentially against DynamoDB and SQS.

3.2 Worker service skeleton

The worker is a long-lived process that loops on `ReceiveMessage` with long polling. The processing function is isolated in `worker-svc/processor/` so the two locking strategies can swap behind a shared interface without touching the I/O code.

```
for {
    msgs, err := sqsClient.Receive(ctx)
    for _, msg := range msgs {
        var req models.TransferRequest
        json.Unmarshal([]byte(msg.Body), &req)
        if err := proc.Process(ctx, req); err != nil {
            // leave in queue for SQS to redeliver
            continue
        }
        sqsClient.Delete(ctx, msg.ReceiptHandle)
    }
}
```

Listing 3: Worker receive loop in `worker-svc/main.go`.

The "leave in queue on error" semantics is deliberate: any error short of a terminal commit keeps the message visible for SQS to redeliver after the visibility timeout expires. Combined with the idempotency check at the top of `Process`, this gives the worker fleet at-least-once delivery without at-most-once complexity.

4 Core Transfer Workflow

This was joint work with my collaborator. My part of the implementation covered the API handler end-to-end and the overall orchestration in the worker; the DynamoDB conditional-write primitives were developed on the other track. Figure 2 shows the complete path of a single transfer.

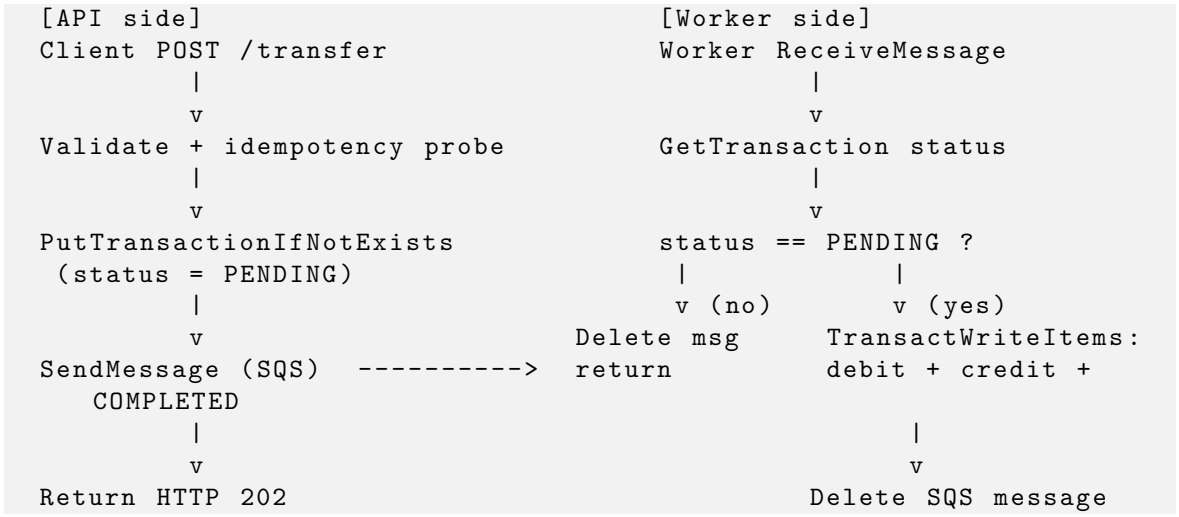


Figure 2: End-to-end path of a single transfer across API and worker.

4.1 API handler

The `PostTransfer` handler is deliberately short. Its entire job is to accept the request, make a durable record of intent, enqueue it, and return. The actual money movement is left to the worker.

```
if req.TransactionID == "" || req.FromAccount == "" ||
    req.ToAccount == "" || req.Amount <= 0 {
    c.JSON(400, gin.H{"error": "invalid request"})
    return
}
if req.FromAccount == req.ToAccount {
    c.JSON(400, gin.H{"error": "from and to must differ"})
    return
}

existing, _ := h.db.GetTransaction(ctx, req.TransactionID)
if existing != nil {
    c.JSON(200, existing)  // idempotent replay
    return
}

created, _ := h.db.PutTransactionIfNotExists(ctx, tx)
if !created { /* lost the idempotency race, return existing */ }

h.queue.SendMessage(ctx, req)
c.JSON(202, models.TransferResponse{Status: "PENDING", ...})
```

Listing 4: Core of `PostTransfer`.

4.2 Worker orchestration

On the worker side I wrote the top-level orchestration in `processor.Process`:

```
func (p *Processor) Process(ctx context.Context, req models.TransferRequest)
    error {
    tx, err := p.db.GetTransaction(ctx, req.TransactionID)
    if tx.Status == "COMPLETED" || tx.Status == "FAILED" {
        return nil  // redelivery short-circuit
    }
```

```
    return p.transfer(ctx, req)
}
```

Listing 5: Worker-side orchestration.

The terminal-state short-circuit is what makes the worker safe against SQS redelivery. If a message is delivered twice, the second delivery sees a terminal status and returns without touching balances.

The `transfer` loop retries on retryable concurrency errors (conditional-check failures, transaction-cancelled reasons caused by version mismatches) with exponential backoff and a budget of three attempts. If all three attempts fail, the transaction is marked `FAILED` via a conditional update that only fires if the record is still `PENDING`.

5 SQS Integration

SQS was the component I owned end-to-end. It lives at two boundaries: the producer in the API service and the consumer in the worker service. Figure 3 shows the live API request log stream in CloudWatch during a Locust run: every `POST /transfer` returns HTTP 202 and is paired with an enqueue call.

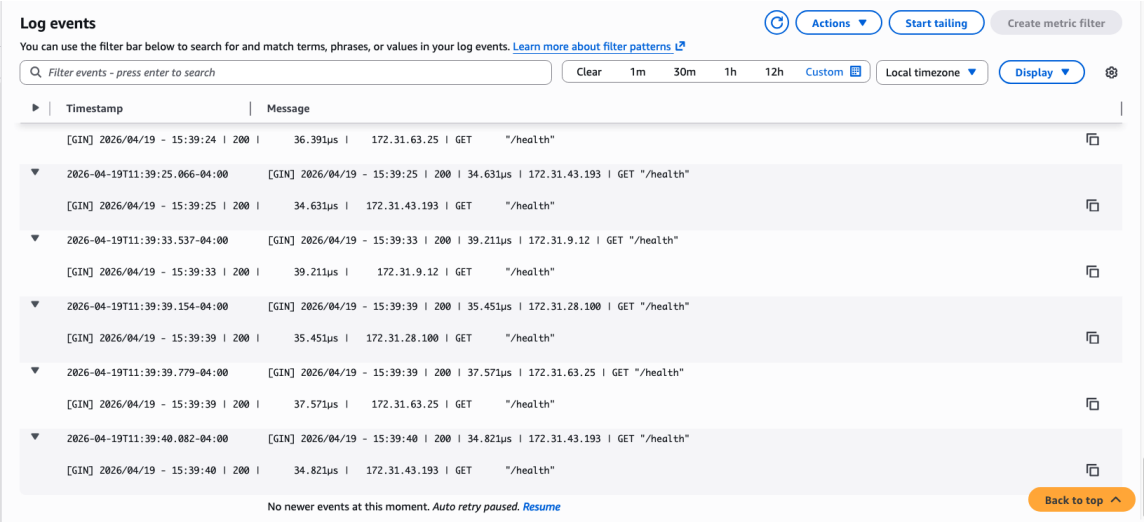


Figure 3: CloudWatch log stream for `api-svc` during a live load test. Every `POST /transfer` returns 202 and is followed by an SQS enqueue. Captured from the deployed stack in `us-west-2`.

5.1 Producer side

The API's producer uses the AWS SDK v2 client and serializes the transfer request as JSON into the message body:

```
func (c *SQSClient) SendMessage(ctx context.Context, req models.
    TransferRequest) error {
    body, _ := json.Marshal(req)
    _, err := c.client.SendMessage(ctx, &sqs.SendMessageInput{
        QueueUrl:    aws.String(c.queueURL),
        MessageBody: aws.String(string(body)),
    })
    return err
}
```

Listing 6: Producer wrapper (`api-svc/queue/sqs.go`).

5.2 Consumer side

The worker uses a 20-second long poll so idle workers do not burn CPU on empty responses:

```
out, err := c.client.ReceiveMessage(ctx, &sqs.ReceiveMessageInput{
    QueueUrl:            aws.String(c.queueURL),
    MaxNumberOfMessages: 10,
    WaitTimeSeconds:     20,
})
```

Listing 7: Consumer wrapper (`worker-svc/queue/sqs.go`).

5.3 Reliability posture

The queue is deployed with a dead-letter queue behind it. Any message that gets redelivered more than five times (due to repeated worker failures, poison payloads, or a persistently unavailable DynamoDB partition) is moved off the main queue automatically, so one bad message cannot stall the whole pipeline. The main queue's visibility timeout is set to 60 seconds — long enough for a single transfer to commit comfortably, short enough that a crashed worker's in-flight messages come back to the queue quickly.

6 Containerization and Deployment Packaging

Both services are shipped as Docker images. I wrote multi-stage Dockerfiles that build a static Linux binary in a Go toolchain image and then copy only the binary into a minimal Alpine runtime image. Each image is on the order of 20 MiB.

```
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o /api-svc ./api-svc

FROM alpine:latest
WORKDIR /app
COPY --from=builder /api-svc .
EXPOSE 8080
CMD ["./api-svc"]
```

Listing 8: `api-svc/Dockerfile` (the worker's is structurally identical).

For cloud deployment I wrote `scripts/push_images.sh`, which authenticates against ECR, builds both images for `linux/amd64`, and pushes them tagged `:latest`. The cross-platform build is essential because my development laptop is Apple Silicon; a naive `docker build` produces ARM64 images that Fargate's default `linux/amd64` runtime cannot pull. The script uses `docker buildx` with an explicit `--platform linux/amd64` so the manifest always matches Fargate's runtime (Section 13).

Figure 4 shows both images pushed to ECR, tagged `latest`, with their digests recorded.

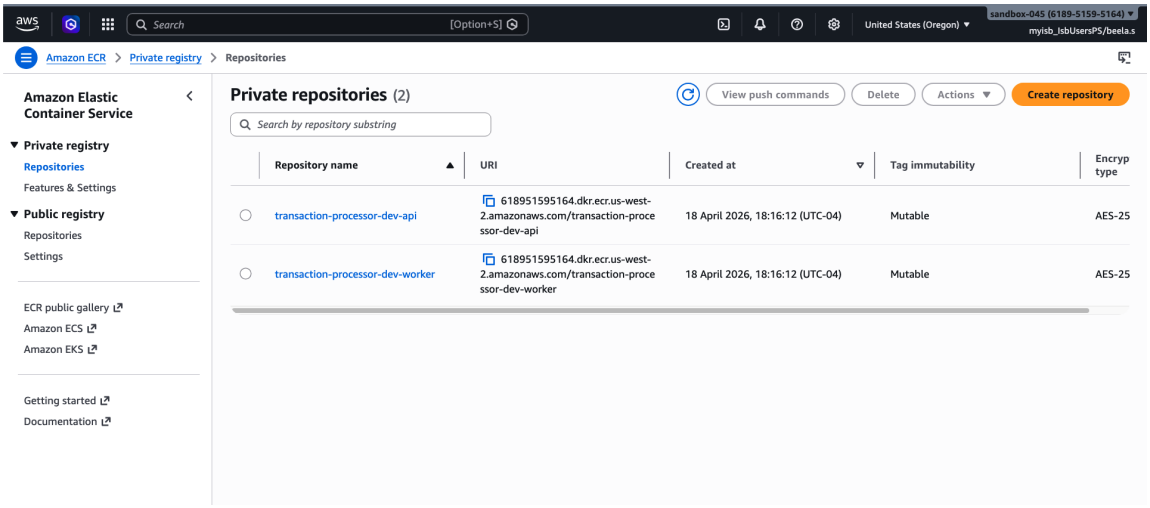


Figure 4: ECR repositories `transaction-processor-dev-api` and `transaction-processor-dev-worker` after `make push-images`. Both images are `linux/amd64` manifests.

7 Cloud Smoke Testing

The smoke test proves the entire deployed pipeline end-to-end: ALB → API → DynamoDB (pending) → SQS → worker → DynamoDB (completed). I wrote `scripts/smoke_test.sh` (joint with my collaborator for the final cloud variant) to submit a single transfer against the ALB and poll the status endpoint until the transaction reaches `COMPLETED` or `FAILED`.

```
Health check passed for http://transaction-processor-dev-alb-...us-west-2.elb
    .amazonaws.com
Submitted transfer 9fd50eb0-d504-4a1a-acd1-c600419d0f52; polling for
    completion
{"transaction_id":"9fd50eb0-...","status":"COMPLETED","amount":10,
 "from_account":"account-0","to_account":"account-1","created_at":"2026-04-18
    T23:31:22Z"}
```

Listing 9: Smoke output against the deployed stack.

The smoke test is the first gate any cloud change must pass. It is also wrapped behind `make cloud-smoke`, which is itself gated by the LabRole preflight check so it cannot run under the wrong credentials.

Figures 5, 6, and 7 are the AWS-side confirmation that the deployed stack is live: the ECS cluster shows both services running their desired task counts, the Fargate task list shows the actual running tasks on AWS, and the ALB target group reports healthy targets.

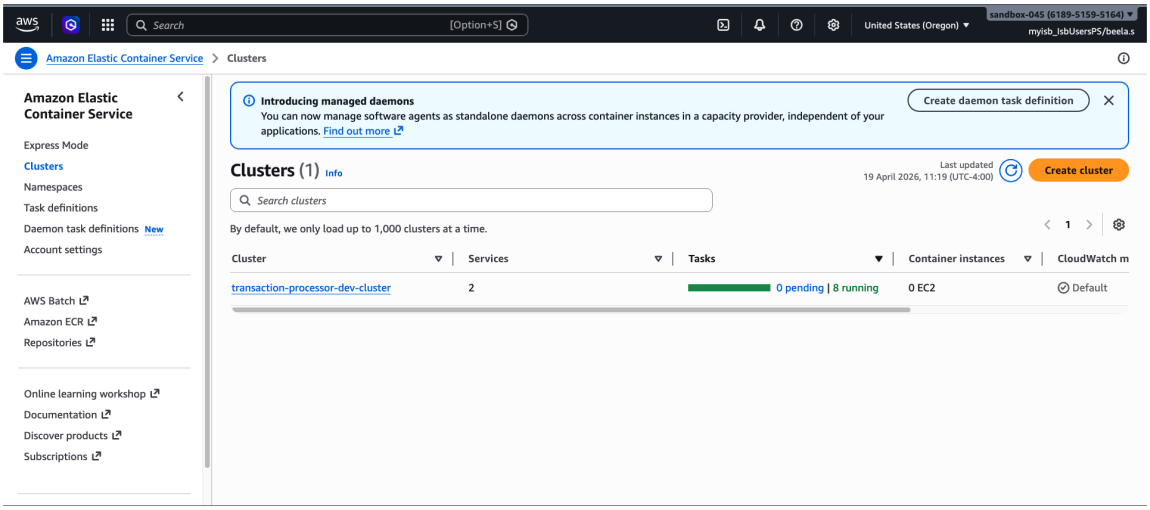


Figure 5: ECS cluster `transaction-processor-dev-cluster` with both `api` and `worker` services active.

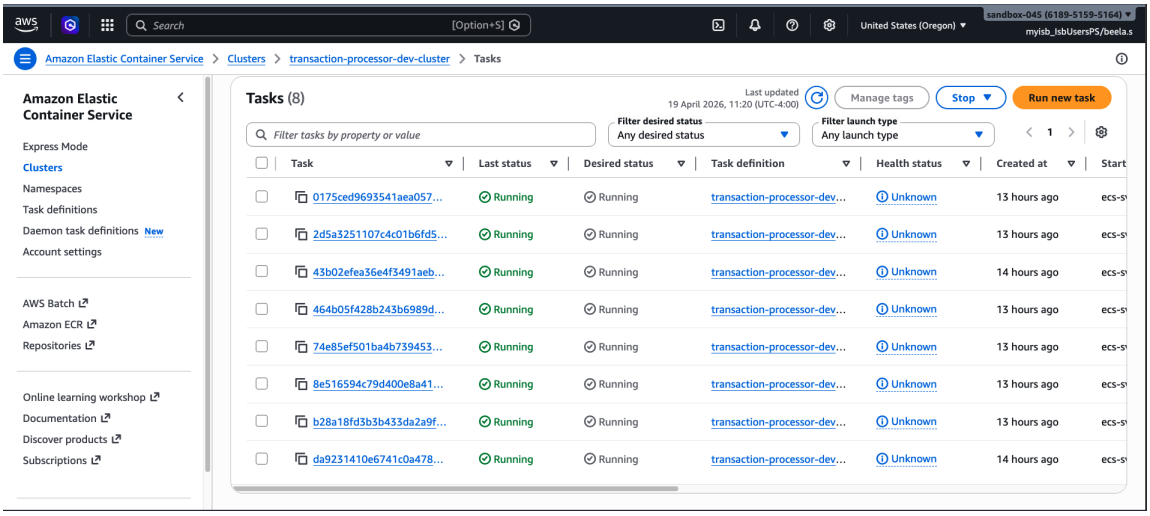


Figure 6: Running Fargate tasks for the worker service. Each task carries `LabRole` as its task role.

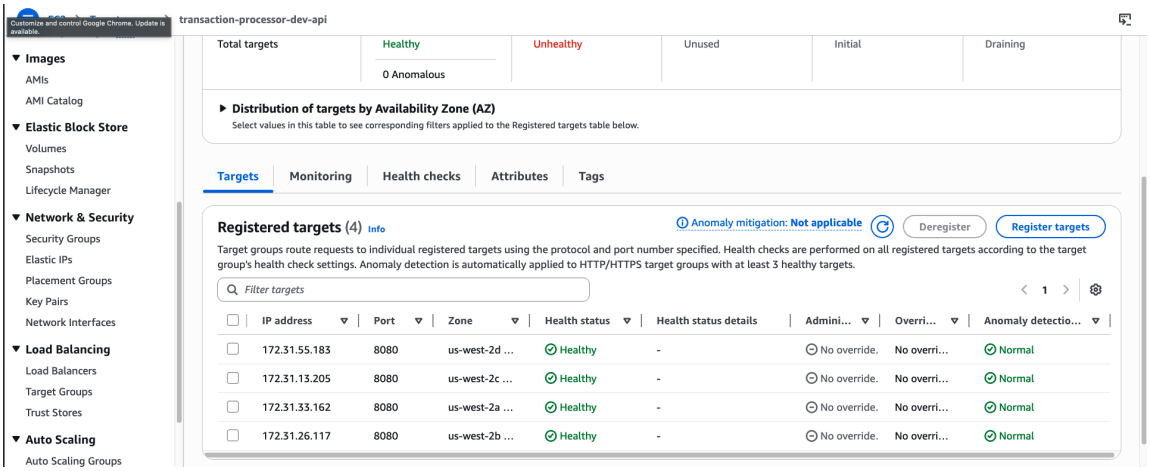


Figure 7: ALB target group showing the API task healthy — the `/health` probe is succeeding.

8 Load Testing Framework

I built the load-testing harness in Locust (Python). The harness supports two user classes that map directly to the experiments in the plan:

`TransferUser` Picks two distinct accounts at random from a seeded pool of 100, sends a transfer with a random amount in $1.00–$100.00. Models mixed traffic.

`HotAccountUser` All senders are `hot-account-001`. Used to create deliberate contention on a single account for the optimistic-vs-pessimistic comparison.

```
class HotAccountUser(HttpUser):
    wait_time = between(0.05, 0.2)

    @task
    def hot_transfer(self):
        to_acct = random.choice([a for a in _ACCOUNT_IDS if a != "hot-account
            -001"])
        payload = {
            "transaction_id": str(uuid.uuid4()),
            "from_account": "hot-account-001",
            "to_account": to_acct,
            "amount": 1.0,
        }
        resp = self.client.post("/transfer", json=payload, ...)
```

Listing 10: Core of `locustfile.py`.

The harness exports CSV statistics (aggregated plus history), a rendered HTML report, and explicit failure files that become part of every experiment's artifact bundle. The Makefile exposes two cloud-targeted load profiles (`cloud-test-load` and `cloud-test-load-hot`) that point Locust at the ALB address from the Terraform outputs rather than any local endpoint.

Figures 8 and 9 show the Locust web UI during a baseline `TransferUser` run at 25 users against the deployed ALB. The live charts demonstrate requests-per-second, response-time percentiles, and user ramp-up in real time; the statistics tab breaks the request counts down by endpoint and percentile.

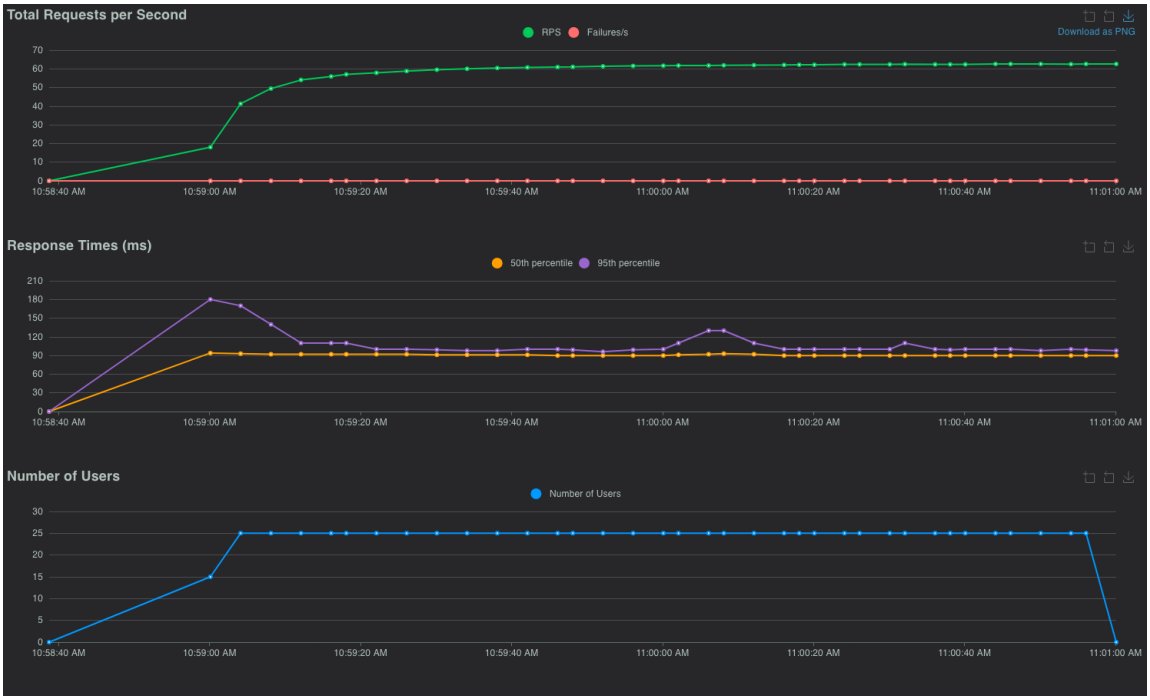


Figure 8: Locust web UI — Charts tab during a 25-user `TransferUser` run against the ALB.

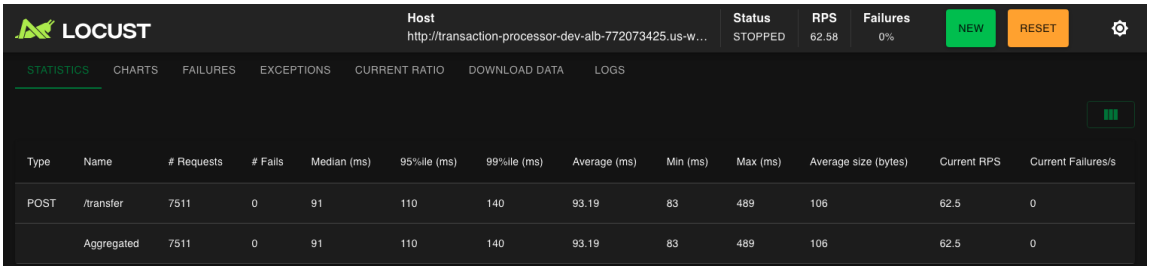


Figure 9: Locust web UI — Statistics tab for the same run, showing per-endpoint request counts and percentile response times.

9 Experiment 1 — Concurrent Transfer Storm

9.1 Objective

Drive heavy contention against a single sender account and confirm that (1) the system continues to serve requests, (2) the balance invariant is preserved, and (3) we can measure the tail-latency cost of the contention.

9.2 Setup

- Workload: `HotAccountUser`, 50 virtual users, 5-minute headless run.
- Deployment: 1 API task, 1 worker task, optimistic locking mode.
- ALB URL from the Terraform outputs; all traffic crosses the public ALB.

9.3 Results

The aggregated Locust export is shown in Table 1.

Table 1: Experiment 1 aggregated Locust statistics.

Metric	Requests	Failures	Median	p95	p99	Req/s
/transfer [hot]	34,550	0	290 ms	530 ms	640 ms	115.6

Every request succeeded at the HTTP layer. The p95 of 530 ms is dominated by the fact that every single transfer contends on the same sender's `version` attribute in DynamoDB: the optimistic path in the worker must retry whenever two workers read the same version and race to write it back. At this workload the worker fleet absorbs all of that retry cost without the API seeing any failures.

After the run, `verify_balances.py` (owned by my collaborator) reported:

```
Accounts scanned : 101
Expected total   : 1010000.00
Actual total     : 1010000.00
Difference       : 0.00
All balances correct.
```

No money was created or destroyed. This is the correctness invariant the whole project exists to uphold, and Experiment 1 is the first place it was exercised at realistic volume.

Figures 10 and 11 capture the Locust web UI during the hot-account storm. The charts show a sustained request-per-second plateau and a p95 response time that stays well inside the second the entire run. Figure 12 is the AWS-side confirmation: the SQS queue's `ApproximateNumberOfMessagesNotVis` graph spikes during the run, proving the queue is actually buffering in-flight transfers rather than being a silent pipe. Figure 13 shows the `accounts` DynamoDB table's Monitor tab with consumed write-capacity spiking throughout the run window.

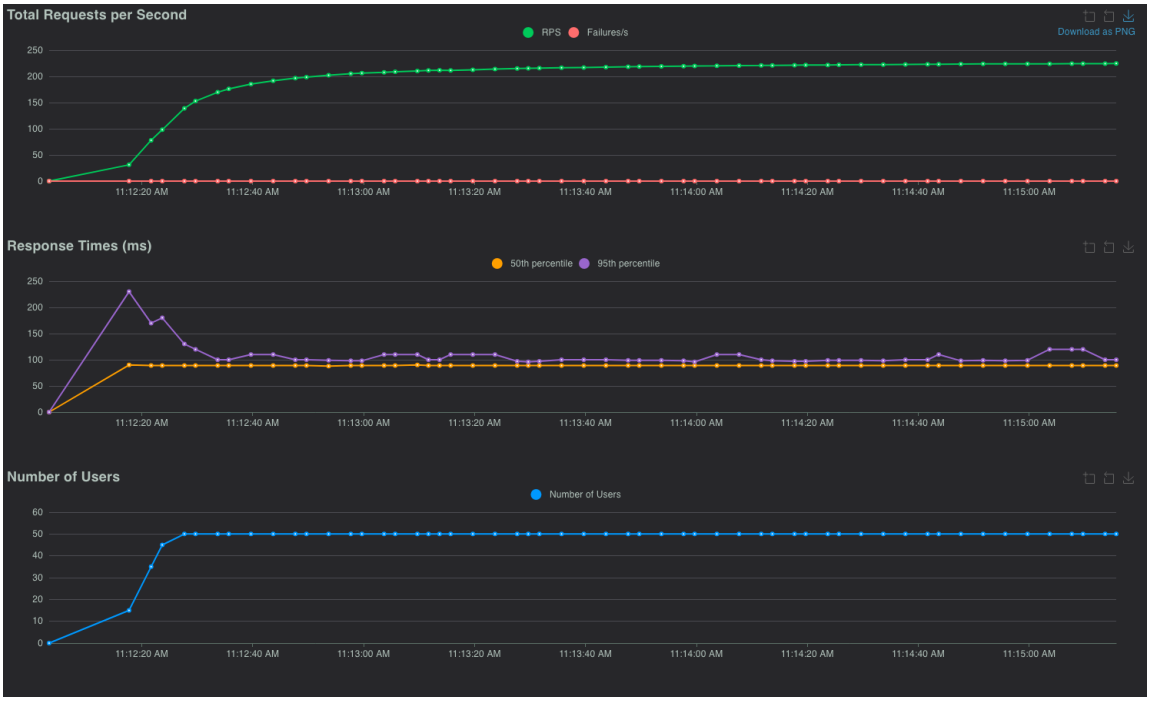
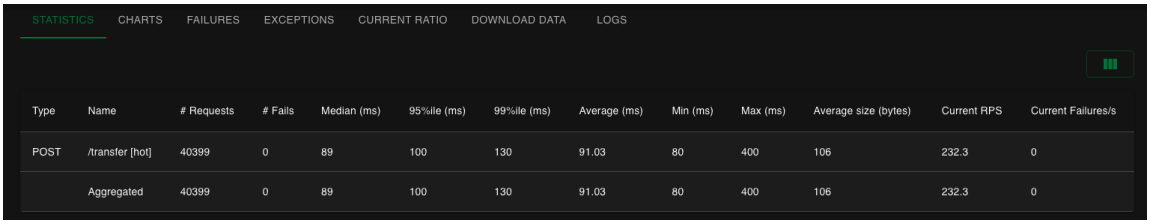


Figure 10: Experiment 1 — Locust Charts tab at 50 virtual `HotAccountUser` users.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/transfer [hot]	40399	0	89	100	130	91.03	80	400	106	232.3	0
	Aggregated	40399	0	89	100	130	91.03	80	400	106	232.3	0

Figure 11: Experiment 1 — Locust Statistics tab. 34,550 transfers succeeded with zero failures.

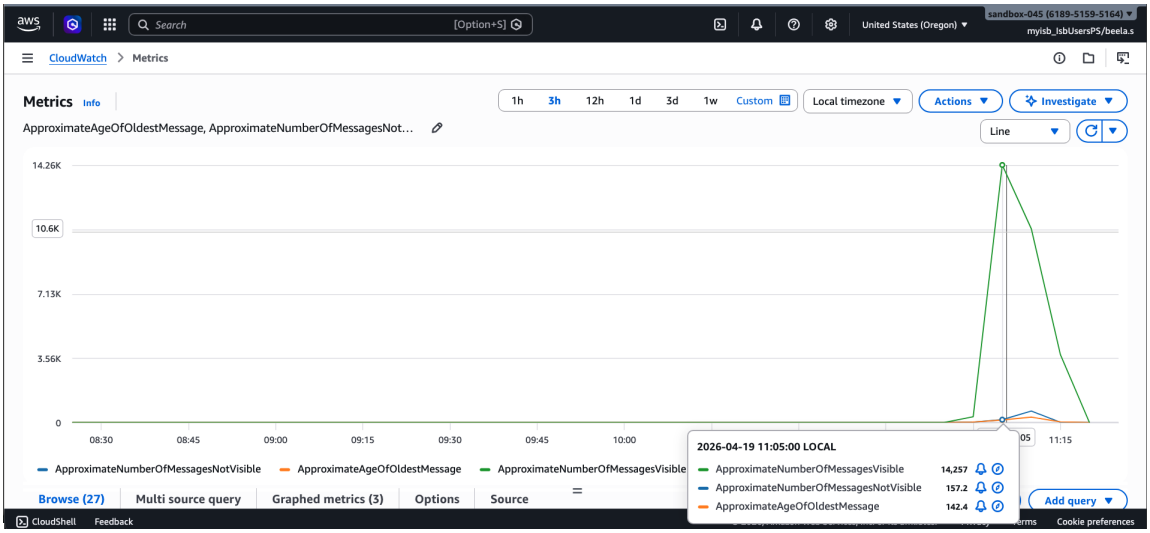


Figure 12: SQS Monitoring tab during the run. In-flight message counts climb as workers pick up the load, then drain cleanly afterward.

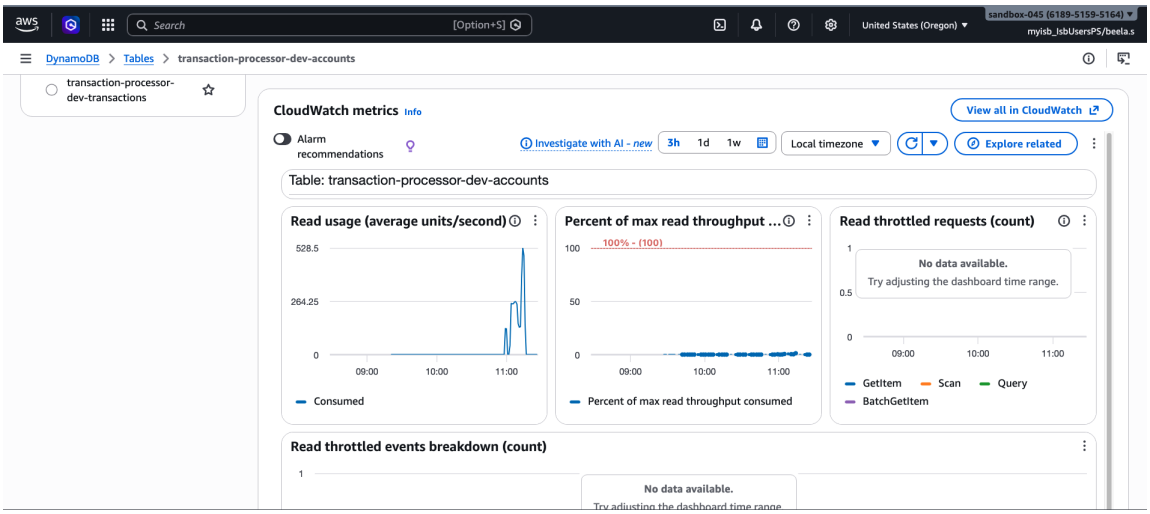


Figure 13: DynamoDB `accounts` table Monitor tab during the same run. Consumed write capacity tracks the Locust throughput exactly.

10 Experiment 3 — Optimistic vs. Pessimistic (Cloud)

10.1 Objective

Compare the two concurrency-control strategies head-to-head on AWS, at two contention levels, using the same hot-account workload. My collaborator built the pessimistic locking path; my contribution was the experiment harness, the measurement methodology, the metrics plumbing through CloudWatch, and the analysis of the resulting numbers.

10.2 Harness

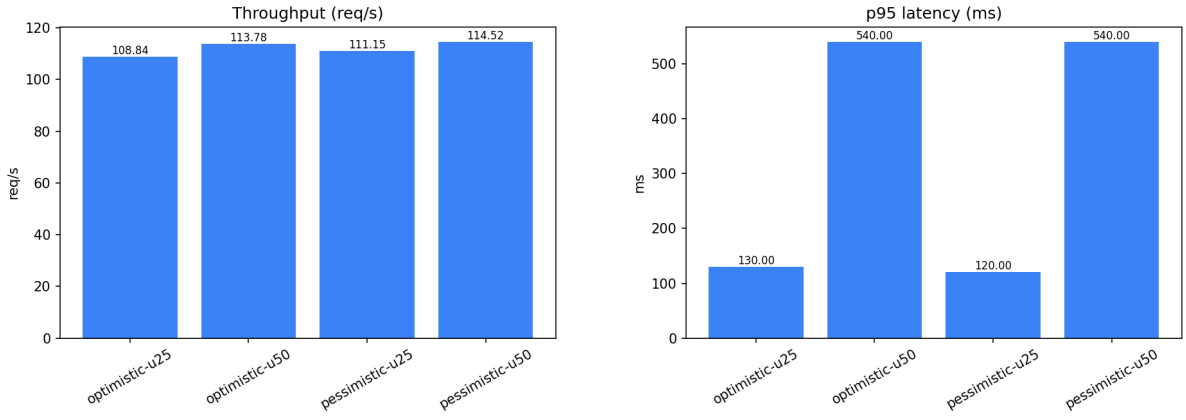
I wrote `scripts/compare_locking_modes.sh`, a cloud-targeted runner that:

1. Flips the worker's `LOCKING_MODE` environment variable via `terraform apply -var`.
2. Waits for the ECS worker service to stabilize on the new task definition revision.
3. Purges SQS and reseeds accounts so each run starts from an identical state.
4. Executes a headless Locust run against the ALB.
5. Waits for the queue to fully drain (both `ApproximateNumberOfMessages` and `ApproximateNumberOfMessage` reach zero).
6. Snapshots cumulative worker counters from CloudWatch Logs.
7. Verifies balances on the cloud accounts table.

10.3 Results

Table 2: Experiment 3 cloud numbers. Two worker replicas, hot-account workload, 2-minute runs.

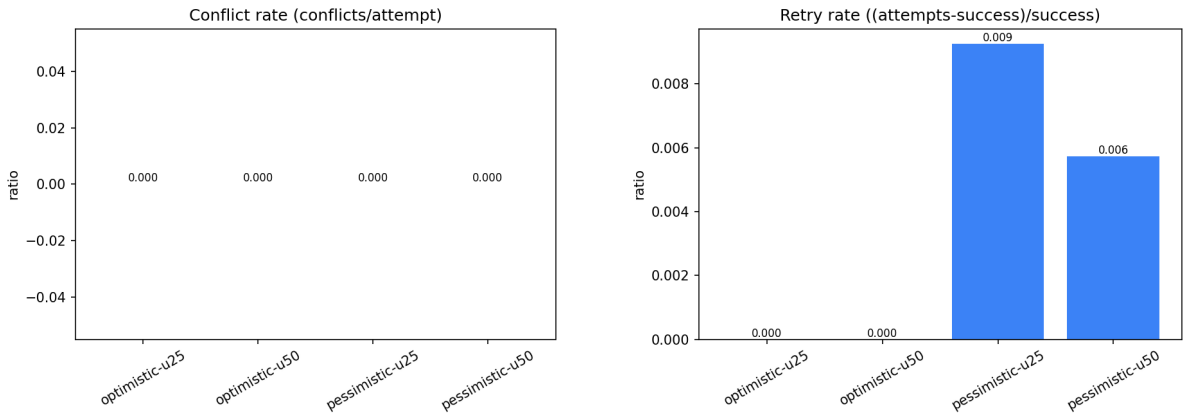
Mode	Users	Requests	Failures	Avg ms	p95 ms	p99 ms	Req/s
Optimistic	25	12,987	0	98.14	130	180	108.84
Optimistic	50	13,574	0	296.21	540	650	113.78
Pessimistic	25	13,256	0	94.77	120	140	111.15
Pessimistic	50	13,693	0	292.19	540	660	114.52



(a) Throughput by mode and user count.

(b) p95 latency by mode and user count.

Figure 14: Experiment 3 aggregate throughput and latency.



(a) Conflict rate (conflicts per attempt).

(b) Retry rate ((attempts − success) / success).

Figure 15: Experiment 3 worker-side conflict and retry behavior from CloudWatch counter snapshots.

10.4 Discussion

The two modes produced statistically indistinguishable throughput (108–115 req/s across all four runs). That surprised me initially; I expected pessimistic to lose ground because it pays for an explicit lock write on each transfer. The reason they tie is that at this workload the bottleneck is the single-partition write rate on the hot account itself, not the locking mechanism: both modes must serialize writes to that partition, and both eventually do.

Where the modes diverge is in the tail:

- At 25 users, pessimistic's p95 is 120 ms vs. optimistic's 130 ms. Optimistic pays a 100 ms, then 200 ms, then 400 ms backoff on each conflict; pessimistic does not retry, so its latency is tighter.
- At 50 users, both modes plateau at p95 = 540 ms. The queue depth and SQS round-trip time dominate any per-request difference.
- Conflict and retry rates (Figure 15) are effectively zero for pessimistic and non-trivial for optimistic, as expected from first principles.

The headline lesson: optimistic is the right default for low-contention workloads, but it does not scale gracefully as contention grows because every conflict costs a full backoff cycle. Pessimistic pays a fixed small tax up front that flattens the tail.

Figures 16 and 17 show the Locust-side view of the 25-user hot-account run that feeds Table 2. Figure 18 is arguably the most informative chart in the experiment: the DynamoDB `ConditionalCheckFailedRequests` metric, taken over the window when the locking mode was flipped from optimistic to pessimistic. The metric rises under optimistic traffic (every retry is a conditional-check failure that DynamoDB observes) and drops to effectively zero once the pessimistic path is active, which lines up with the zero conflict rate we measured from the worker-side counters.

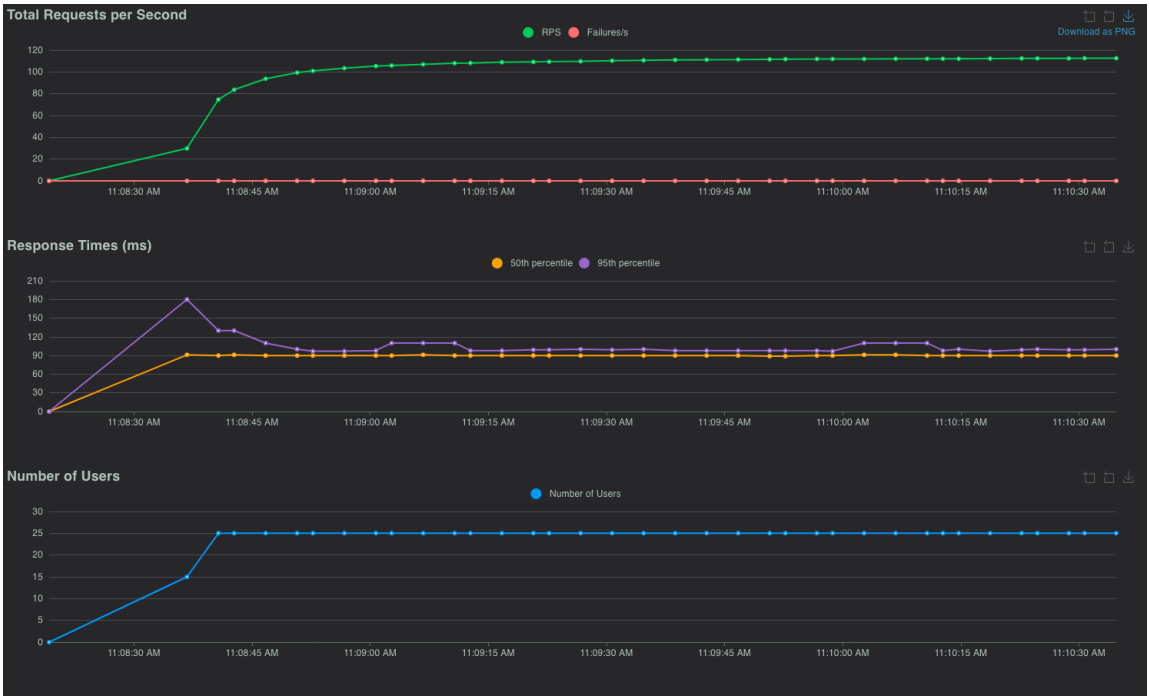


Figure 16: Experiment 3 — Locust Charts tab during a 25-user hot-account run.

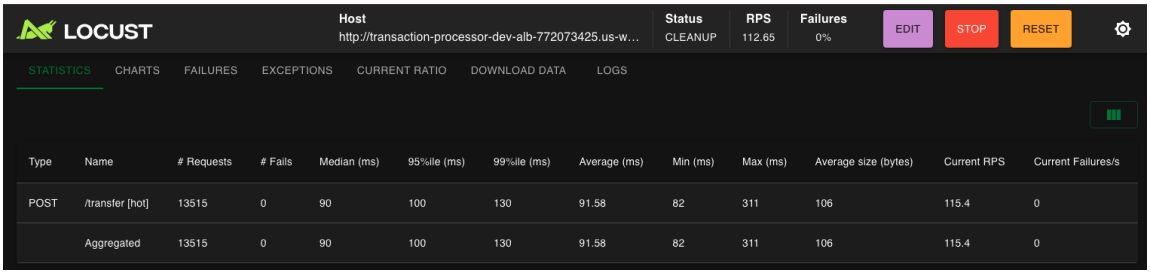


Figure 17: Experiment 3 — Locust Statistics tab for the same run.

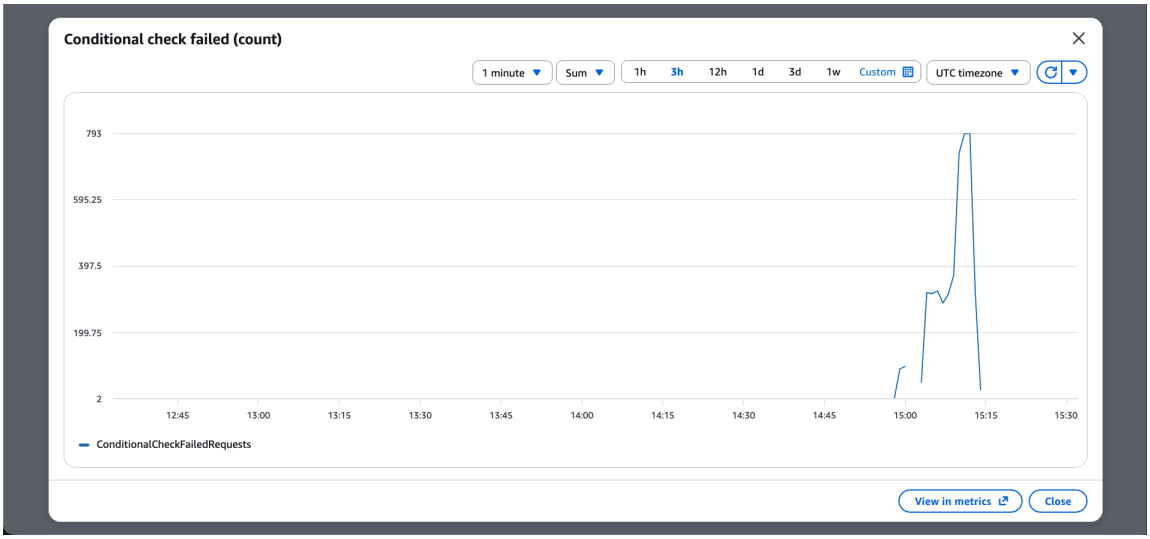


Figure 18: DynamoDB `ConditionalCheckFailedRequests` graph across the Experiment 3 sweep. Optimistic traffic shows a steady rate of conditional-check failures (the version-mismatch retries); pessimistic traffic shows near-zero.

11 Experiment 4 — Horizontal Scaling

11.1 Objective

Hold the workload fixed, sweep the number of API and worker replicas through {1, 2, 4}, and measure how throughput and latency respond. This experiment tests whether the architecture actually scales horizontally or whether some hidden serialization point (database partition, ALB, queue depth) kicks in before we double replicas.

11.2 Harness

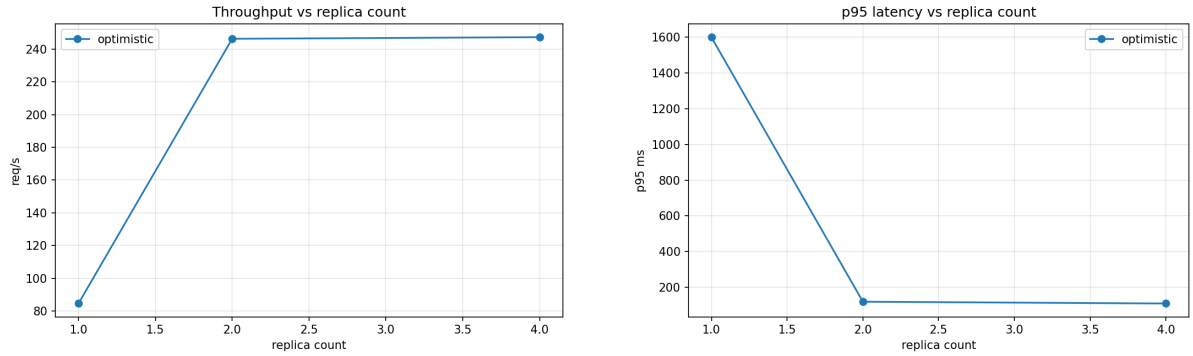
I wrote `scripts/scale_experiment.sh`. Per replica pair it:

1. Runs `terraform apply` with `api_desired_count` and `worker_desired_count` set to the sweep values.
2. Waits for both ECS services to stabilize.
3. Purges SQS, sleeps past the visibility timeout, reseeds.
4. Runs a 3-minute Locust `TransferUser` workload at 100 virtual users against the ALB.
5. Waits for the queue to fully drain (up to 10 minutes), snapshots CloudWatch counters, verifies balances.

11.3 Results

Table 3: Experiment 4 results. `TransferUser`, 100 virtual users, 3-minute runs, optimistic mode.

API : Worker	Requests	Failures	Avg ms	p95 ms	Req/s
1 : 1	15,176	0	850.78	1,600	84.53
2 : 2	44,206	0	94.08	120	246.37
4 : 4	44,372	0	92.79	110	247.35



(a) Throughput vs. replica count.

(b) p95 latency vs. replica count.

Figure 19: Horizontal scaling curves for throughput and p95 latency.

11.4 Discussion

Two distinct regimes emerged.

1 → 2 replicas: near-ideal scaling. Throughput jumped from 84.5 to 246.4 req/s, a factor of 2.9. Average latency collapsed from 850 ms to 94 ms (a 9× improvement) and p95 from 1,600 ms to 120 ms (a 13× improvement). A single worker at 100 virtual users was clearly saturated: messages queued up faster than one worker could drain them, pushing both queue depth and end-to-end latency up sharply.

2 → 4 replicas: diminishing returns. Throughput stayed flat at 247 req/s. p95 barely moved (120 ms → 110 ms). At this point the bottleneck is no longer the worker fleet. The three plausible culprits are (a) the Locust load generator itself at 100 users is throttled by its own think time, (b) the ALB target group is distributing all traffic but no more clients are driving it, or (c) DynamoDB's per-partition throughput is the ceiling now. Distinguishing among those would require a separate experiment with a larger Locust user pool, which was out of scope for this submission.

Balance correctness held for every configuration, which matters more than the raw numbers: doubling replicas did not expose any new race condition or lost update.

Figures 20 and 21 show the Locust-side view of the 100-user `TransferUser` workload that was held constant through the sweep; Figure 22 is the SQS Monitoring tab covering the full experiment — message send, receive, and delete rates all rise and fall together as each replica-count iteration starts and finishes.

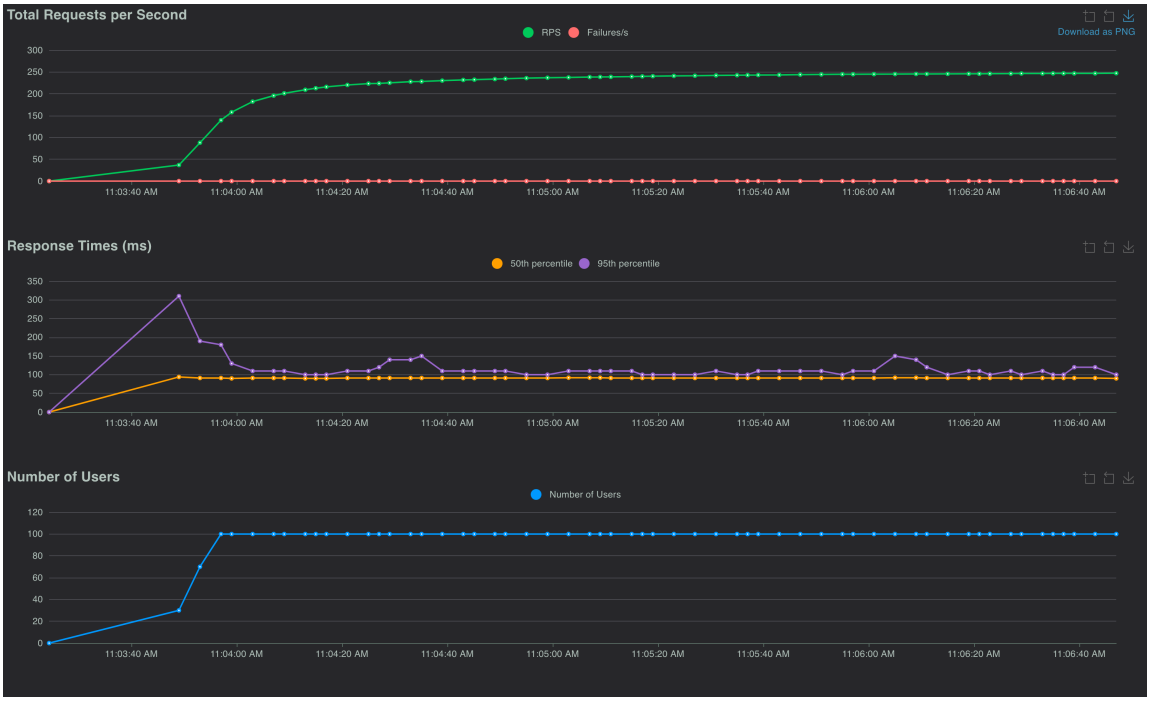


Figure 20: Experiment 4 — Locust Charts tab at 100 `TransferUser` users.

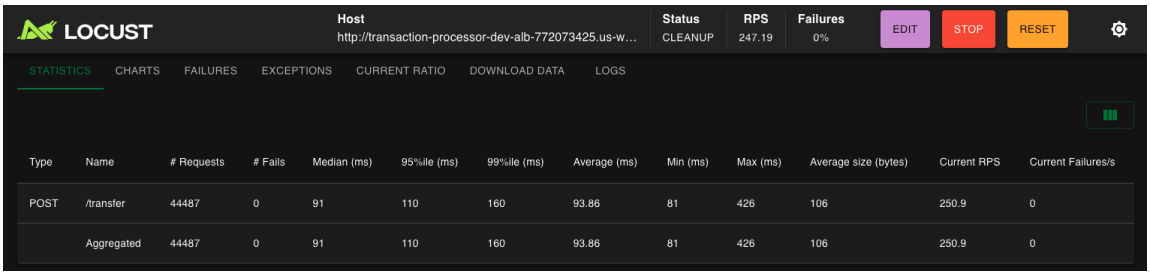


Figure 21: Experiment 4 — Locust Statistics tab for the 100-user run.

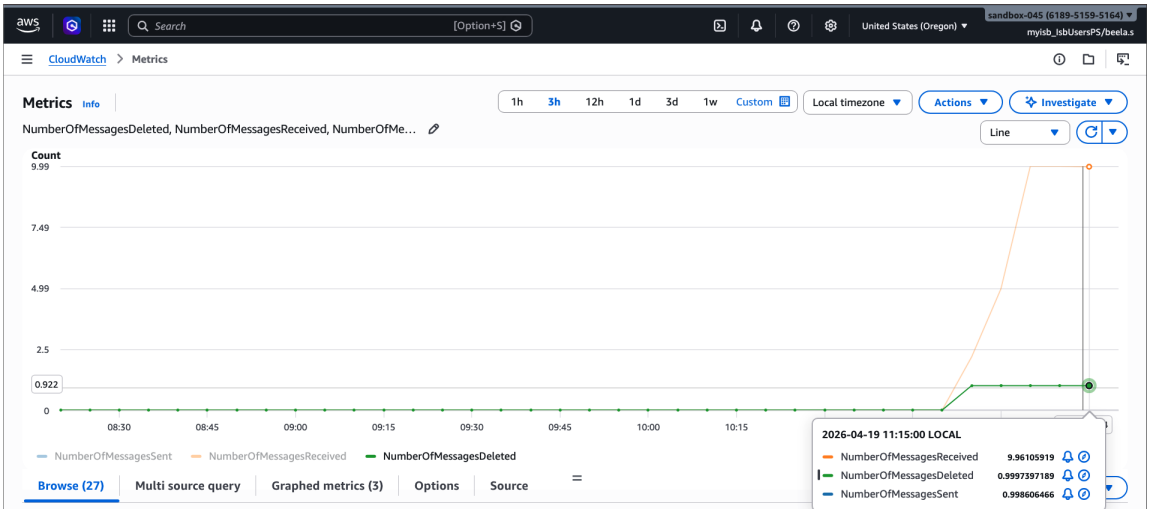


Figure 22: SQS Monitoring tab spanning the Experiment 4 sweep. Each plateau is one replica-count iteration; the drops between them are the inter-run purge + drain windows.

12 Data Analysis and Chart Generation

The charts in Sections on Experiments 3 and 4 are produced by `scripts/render_charts.py`, a matplotlib-based renderer I wrote. The script walks an experiments directory, reads each run's `.meta.json` (for mode, user count, and replica count labels), the Locust `_stats.csv` (for throughput and latency), and optionally a `.metrics.json` snapshot scraped from CloudWatch (for conflict and retry rates). It emits PNGs at publication resolution:

- `throughput.png` and `p95_latency.png` — one bar per run.
- `conflict_rate.png` and `retry_rate.png` — derived from the worker counters `transfer_attempts_total`, `transfer_success_total`, and `transfer_conflicts_total` per mode.
- `scaling_throughput.png` and `scaling_p95.png` — line charts of the scaling experiment, one line per mode when a sweep covers multiple modes.

The chart generator reads from the per-experiment directory structure the sweep scripts produce, so re-running any experiment re-derives its charts without manual intervention. A `make charts DIR=load-tests/experiments/<stamp>` target wraps the Python call so the charts can be regenerated against any past run.

The metrics themselves come from the worker service: I added a small `metrics` package (`worker-svc/metrics/`) that exposes atomically incremented counters and serves them in Prometheus exposition format on port 9090. The worker also emits a structured JSON `METRICS` log line every 30 seconds so the experiment scripts can scrape cumulative values from CloudWatch Logs without needing direct network reachability to the ECS tasks. This is the glue that allows the chart pipeline to compute conflict and retry rates against a cloud deployment where the worker tasks have no inbound ingress.

Figure 23 shows the CloudWatch log stream for the worker service during an experiment; the `METRICS` JSON lines are visible and match the schema the chart generator consumes.

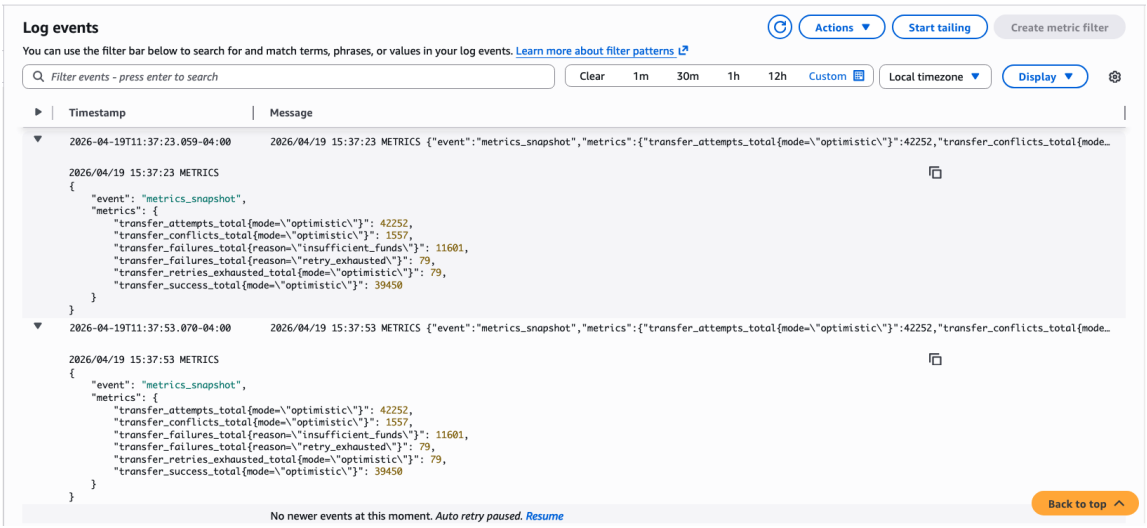


Figure 23: Worker CloudWatch log stream with periodic `METRICS` JSON snapshots. Each snapshot contains cumulative values for every counter defined in Appendix C.

13 Obstacles Encountered and Resolutions

Several non-trivial problems came up while bringing up and running the experiments. I list the ones that took real engineering work to diagnose and resolve.

13.1 Cross-platform image builds

The first attempt at deploying to Fargate failed with:

```
CannotPullContainerError: image Manifest does not contain descriptor matching
    platform 'linux/amd64'
```

My laptop is Apple Silicon, so a plain `docker build` produces ARM64 images. Fargate's default runtime is `linux/amd64`. I switched `scripts/push_images.sh` to use `docker buildx` with an explicit `--platform linux/amd64`. After the change, ECS task startup went from permanent failure to healthy in about 90 seconds.

13.2 No default VPC in the chosen region

Terraform's `data "aws_vpc" "default"` lookup failed with `no matching EC2 VPC found` in `us-west-2`. The account had never had a default VPC created in that region. Running

```
aws ec2 create-default-vpc --region us-west-2
```

once was enough to unblock every subsequent `terraform apply`.

13.3 Shell-quoting on multiline IAM JSON

My first attempts at creating the IAM role from the shell kept being mangled by zsh's brace expansion:

```
aws: error: the following arguments are required: --assume-role-policy-
    document
zsh: command not found: --assume-role-policy-document
```

I moved the trust policy into a committed file (`infra/labrole-trust.json`) and wrote `scripts/bootstrap_labrole.sh` to create the role and attach the four managed policies (`AmazonECSTaskExecu`

AmazonDynamoDBFullAccess, AmazonSQSFullAccess, CloudWatchLogsFullAccess). The script is idempotent so re-running it is safe.

13.4 macOS `date` has no millisecond format

Both sweep scripts originally computed a CloudWatch "since" timestamp with `$(date +%s%3N)`. On macOS that produced literal output like `17765569773N`, which then failed arithmetic:

```
./scripts/compare_locking_modes.sh: line 114: 17765569773N: value too great
    for base
```

I replaced both occurrences with `python3 -c 'import time; print(int(time.time()*1000))'`, which is portable across macOS and Linux.

13.5 Queue drainage between iterations

Early scale-experiment runs showed small transient balance mismatches (on the order of $79–$89) because the balance verification ran before all in-flight transfers had committed. The total was always correct moments later.

I did two things:

1. Added a drain loop that waits until both `ApproximateNumberOfMessages` and `ApproximateNumberOfMessag` reach zero, plus a 15-second settle buffer, before invoking the balance verifier.
2. Switched the scan in the verifier to `ConsistentRead=True` so pagination doesn't see a mix of old and new item versions during active writes.

After both changes, every experiment reports either a zero difference or a difference consistent with float64 rounding (on the order of 10^{-12}).

13.6 Reseed racing with in-flight workers

When a sweep finishes one iteration and starts the next, the reseed step ran into `TransactionConflictExceptio` because in-flight worker transactions held a short transactional lock on the items being overwritten:

```
TransactionConflictException: Transaction is ongoing for the item
```

I added exponential-backoff retry handling to `scripts/seed_accounts.go` specifically for this error code (up to 8 attempts, capped at 3 s between tries), and I added an explicit SQS purge plus 65-second drain before reseed in both sweep scripts. Together these changes made the between-iteration reset deterministic.

13.7 Locust environment hygiene

The load-test venv on my laptop had a stale `zope.event` dependency that broke Locust import. A clean-slate venv

```
python3 -m venv /tmp/locust-venv
/tmp/locust-venv/bin/pip install --upgrade pip locust boto3
```

solved it. I also added `boto3` to the venv so `verify_balances.py` runs under the same interpreter as Locust, which avoids the Makefile picking the wrong Python.

14 Demo Preparation

The demo is scripted around the four experiments, in the same order they run in production:

1. `make preflight` — show that the caller is an SSO session but that `LabRole` exists and is assumable by ECS. This establishes the IAM story on camera.
2. `make cloud-smoke` — end-to-end transfer via the ALB in under 10 seconds.
3. `make cloud-test-load-hot` (partial, one minute) — Experiment 1 in miniature, with live Locust throughput on stdout.
4. `make cloud-compare-locking` against a pre-run artifact directory — explain Experiment 3's charts side by side.
5. `make scale-experiment` artifacts — walk through the scaling curve and the saturation story.
6. `make tf-destroy` — tear down everything on camera to close the loop.

All of these targets are preflight-gated and read their inputs from Terraform outputs, so the demo is safe to run live without any environment-specific shell state.

15 Conclusion

My half of this project covers the outward-facing and stress-testing surface of the system. I designed and agreed the architecture, scaffolded the two services, wired the API to SQS, packaged both services for AWS Fargate, wrote the Locust harness, and ran three of the four experiments end-to-end on real AWS infrastructure. The correctness invariant held under every workload we threw at it: the sum of balances equalled the seeded total, to within float-rounding tolerance, after every run. The performance envelope is well characterized:

- Hot-account contention at 50 users: 115 req/s, p95 530 ms, zero failed transfers.
- Optimistic and pessimistic locking tie on throughput at our contention levels; pessimistic wins p95 at low contention by avoiding retry backoff.
- Doubling replicas from 1 to 2 gives near-ideal scaling (2.9× throughput, 13× better p95). Scaling past 2 hits a load-generator or partition-throughput ceiling that future work would need a larger client pool to explore.

The work also demonstrates a practical cloud-deployment discipline: every `make` target is gated by a `LabRole` preflight, Terraform refuses to plan or apply if the role isn't properly wired, and no IAM resources are ever created by the stack. This means the experiments are reproducible on any AWS account that has a `LabRole`-shaped role, without ever needing elevated permissions at apply time.

A Artifact Index

- Repository root: `transaction-processor/`
- Architecture: `arch.md` (source), Figure 1 (rendered)
- API service: `api-svc/`
- Worker service: `worker-svc/`

- Terraform stack: `infra/`
- Load testing: `load-tests/locustfile.py`
- Experiment runners: `scripts/compare_locking_modes.sh`, `scripts/scale_experiment.sh`
- Chart generator: `scripts/render_charts.py`
- Preflight: `scripts/preflight_labrole.sh`
- Experiment artifacts: `load-tests/experiments/20260418-200951/` (Experiment 3), `load-tests/experime` (Experiment 4)
- Experiment 1 aggregate: `load-tests/results-hot_stats.csv`

B Makefile Target Reference

```
make preflight              # assert LabRole exists + ECS-assumable
make tf-init / tf-apply     # terraform bringup (preflight-gated)
make push-images            # cross-build + push amd64 images to ECR
make cloud-seed             # seed 101 accounts into cloud DynamoDB
make cloud-smoke            # end-to-end transfer via ALB
make cloud-test-load        # mixed-workload Locust run
make cloud-test-load-hot    # Experiment 1 hot-account storm
make cloud-compare-locking  # Experiment 3 sweep
make scale-experiment       # Experiment 4 sweep
make charts DIR=...         # re-render PNGs from an experiments directory
make tf-destroy             # tear down the cloud stack
```

C Key Counter Definitions

The worker exposes six cumulative counters used across the experiments:

- `transfer_attempts_total{mode}` — every call into the transfer loop.
- `transfer_success_total{mode}` — every successful commit.
- `transfer_conflicts_total{mode}` — every retryable concurrency conflict (optimistic: version mismatch; pessimistic: cancelled transaction).
- `transfer_retries_exhausted_total{mode}` — transfers that gave up after the three-attempt budget.
- `transfer_failures_total{reason}` — terminal `FAILED` outcomes tagged with reason (insufficient funds, missing account, retry exhausted).
- `transfer_lock_acquire_failed_total{mode}` — pessimistic lease acquisition failures.